Arduino IDE
und ESP32forth

Marc PETREMANN

Version 1.1 - 16. Mär 2026

Inhaltsverzeichnis

Präambel 4
Installieren Sie die Arduino IDE 5
Installation unter Windows 5
ARDUINO IDE 5
ESP32forth herunterladen und vorbereiten 7
Dateien für ESP32forth vorbereiten 7
Projekt öffnen 8
Auswahl einer bestimmten Karte 8
Auswahl einer bestimmten Karte 11
Die Werkzeuge 13
Auto Format 13
Archive Sketch 14
Manage Libraries 14
Serial Monitor 16
Board 16
Bedeutung für den ESP32 16
Beispiele gängiger ESP32-Boards 17
Konfiguration eines ESP32-Boards 17
Port 18
Get Board Info 18
CPU Frequency 19
Core-Debug-Level 20
USB DFU On Boot 21
Erase All Flash Before Sketch Upload 22
Events Run On 23
Flash Frequency 24
Flash Mode 25
Flash Size 26
Gängige Größen für den ESP32 27
JTAG-Adapter 27
Arduino Runs On 28
Partition Scheme 29
Die gängigsten Optionen 30
Zusammenstellungsbericht anzeigen 31
PSRAM 31
Grenzen, die zu beachten sind 32
Upload Speed 32
Warum sollte man sich für die eine Geschwindigkeit und gegen die andere entscheiden? 32
USB CDC On Boot 33
Zigbee Mode 33
Die verschiedenen verfügbaren Modi 34
ESP32-Platinen 35
Die gängigsten Kartenformate 35
ESP32 WROOM 36
Das Herzstück des Systems (Mikrocontroller) 36
Drahtlose Konnektivität 36
Eingänge und Ausgänge (GPIO) 37
Energieverbrauch 37
Stromversorgung 37

Parameter tools für ESP32-WROOM....37
Parameter tools für ESP32-WROVER....38
ESP3 C3 Zero....38
Technische Spezifikationen (Hardware)....38
Schnittstellen und Pins (E/A)....39
Empfohlene Einstellungen in der Arduino IDE....39
Wichtige Punkte....39
Parametertools für ESP32 C3 Zero....40
ESP32 S3 WROOM / Entwicklermodul....40
Rechenleistung und KI....40
Konnektivität....40
Die verschiedenen Versionen....41
Parametertools für ESP32-S3-WROOM....41
Parametertools für ESP32-S3-WROOM-N16R8....42
Ressourcen....43
Forth....43

Präambel

Dieses Handbuch soll Ihnen helfen zu verstehen, wie Sie mit dem ARDUINO IDE-Programm ESP32forth für EPS32-Boards und im Übrigen auch alle anderen Programme in der Programmiersprache C für die verschiedenen Boards der ARDUINO-Umgebung kompilieren können.

Installieren Sie die Arduino IDE.

Laden Sie die neueste Version der ARDUINO IDE von der offiziellen Website herunter: https://www.arduino.cc/en/software/

Wählen Sie das Zielsystem aus und klicken Sie anschließend auf *Download*.

Nach dem Herunterladen befolgen Sie bitte die für Ihr System spezifischen Anweisungen. (Windows, Linux, MacOS…).

Installation unter Windows

Hier hat die heruntergeladene Version den Dateinamen *arduino-ide_2.3.7_Windows_64bit.exe*.

Nach dem Herunterladen sollte sich die Datei im Ordner *Downloads* befinden.

Um die Installation zu starten, führen Sie diese Datei aus. Folgen Sie den Anweisungen des Installationsvorgangs.

ARDUINO IDE

Dans le contexte d'Arduino, le sigle **IDE** signifie **Integrated Development Environment**, ce qui se traduit en français par **Environnement de Développement Intégré**.

Im Kontext von Arduino steht das Akronym **IDE** für **Integrated Development Environment**, was auf Deutsch Integrierte Entwicklungsumgebung heißt.

Dies ist das unverzichtbare Werkzeug, das als "Kommandozentrale" für die Erstellung Ihrer Elektronikprojekte dient.

Die IDE ist ein einzelnes Softwarepaket, das alle Werkzeuge vereint, die ein Programmierer benötigt, um eine Idee in Code und anschließend auf einem Arduino-Board auszuführen. Sie besteht im Wesentlichen aus drei Teilen:

- **Der Texteditor**: Hier schreiben Sie Ihren Code (in Arduino als „Sketch" bezeichnet). Er hebt Schlüsselwörter hervor, um Ihnen zu helfen, Syntaxfehler zu vermeiden.
- **Der Compiler**: Er übersetzt Ihren Code (für Menschen lesbar) in Maschinensprache (0 und 1), die der Mikrocontroller auf der Platine versteht.
- **Der Programmierer (Uploader)**: Er sendet den kompilierten Code über das USB-Kabel an Ihr ESP32-Board.

Vor der Einführung von IDEs mussten Entwickler separate Software verwenden: eine zum Schreiben, eine zum Kompilieren und eine weitere zum Hochladen der Datei auf den Chip. Die Arduino IDE integriert all dies in eine einfache Benutzeroberfläche und macht Elektronik so auch für Anfänger zugänglich.

Die beiden Hauptversionen:

- Arduino IDE 1.8.x : Die klassische Version, sehr stabil und leicht.
- Arduino IDE 2.x : Die moderne Version mit Autovervollständigung (sie errät, was Sie schreiben werden) und einem eingebauten Debugger.

ESP32forth herunterladen und vorbereiten

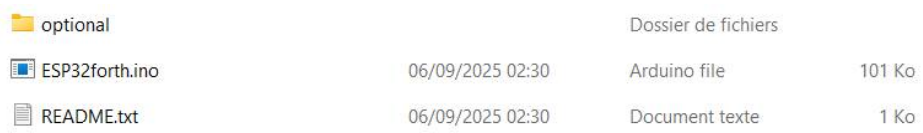
Um ESP32forth herunterzuladen, gehen Sie hierher:
https://esp32forth.appspot.com/ESP32forth.html

Bevorzugt wird Version 7.0.7.22. Diese Version ist mit der ARDUINO IDE 2.x kompatibel.

Dateien für ESP32forth vorbereiten

Öffnen Sie die ZIP-Datei mit ESP32forth und extrahieren Sie deren Inhalt in Ihren Entwicklungsarbeitsbereich. Es empfiehlt sich, dafür einen anderen Ort als Ihren Windows-Desktop zu wählen.

Nach dem Entpacken sollte ein Ordner namens ESP32forth mit folgenden Dateien vorhanden sein:

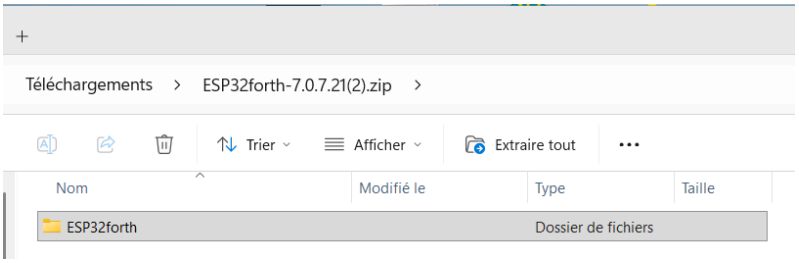


Falls Sie ESP32forth noch nie kompiliert und installiert haben, lassen Sie den Inhalt dieser Ordner unverändert.

Falls Sie noch nie für Arduino, ESP32 oder eine andere elektronische Platine entwickelt haben, erstellen Sie einen Ordner namens **Entwicklungen** in Ihrem Windows-Benutzerbereich:

`c : → Utilisateurs → myProfil → developpements → ESP32`

Erstellen Sie in diesem Entwicklungsordner einen weiteren Ordner namens **ESP32**. Kopieren Sie den Inhalt der zuvor heruntergeladenen Zip-Datei in diesen Ordner :

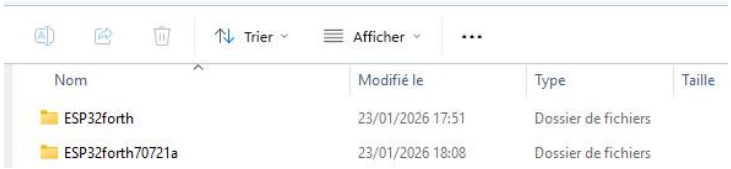


Falls Sie weitere Codes für ESP32 erstellen müssen, definieren Sie andere Verzeichnisse im ESP32-Ordner.

Projekt öffnen

Öffnen Sie die Arduino IDE. Öffnen Sie anschließend das zuvor heruntergeladene Projekt. Klicken Sie dazu auf *File* und wählen Sie *Open*. Suchen Sie das Projekt ESP32forth. Wählen Sie die Datei **ESP32forth.ino** aus. Es erscheint folgendes Fenster:

WICHTIG: das Projekt muss denselben Namen wie der Ordner haben, in dem es sich befindet. Wenn Sie beispielsweise eine Kopie dieses Projekts erstellen möchten, um Änderungen vorzunehmen, erstellen Sie einen neuen Ordner namens **ESP32forth70721a** und legen Sie die Kopie der Datei **ESP32forth.ino** in diesen neuen Ordner. Benennen Sie anschließend den Ordner **ESP32forth** in **ESP32forth70721a** um :



Sie können Änderungen in diesem neuen Ordner vornehmen, ohne den Inhalt des Ordners **ESP32forth** zu verändern.

In diesem Stadium müssen wir einige Parameter festlegen, bevor wir unser Projekt auf das ESP32-Board hochladen können.

Auswahl einer bestimmten Karte

Bevor die Liste der Karten angezeigt werden kann, muss die IDE wissen, was ein ESP32 ist.

- Gehe zu **> Preferences**.
- Im feld **Additionnal boards manager URLs**, fügen Sie diesen Link ein: `https://dl.espressif.com/dl/package_esp32_index.json`

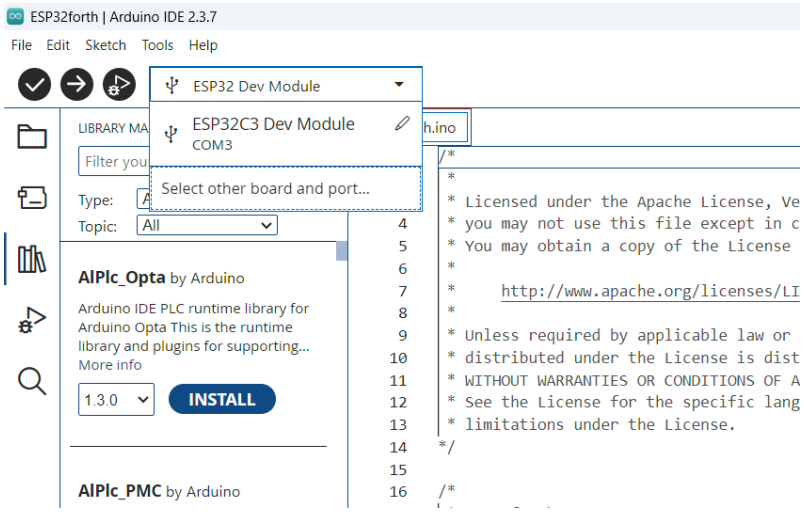
- Als Nächstes gehen Sie zu **Tools > Board > Boards Manager**, suchen **ESP32** und klicken Sie auf **Instal**.

Sobald das Paket installiert ist, gehen Sie wie folgt vor, um Ihr Modul auszuwählen:

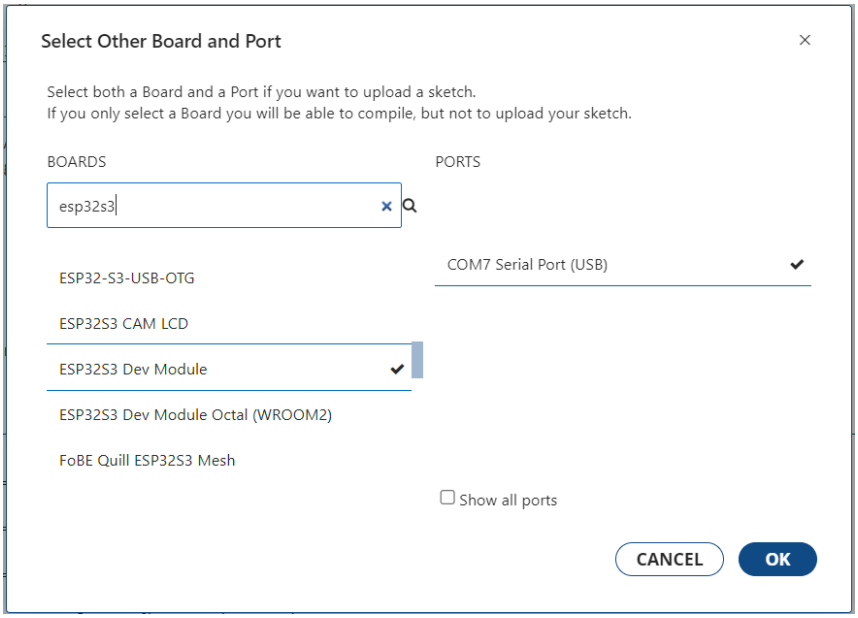
1. Menü öffnen **Tools**.
2. Platzieren Sie Ihre Maus auf **Board : "..."** (oft standardmäßig "Arduino Uno").
3. Suchen Sie nach dem Abschnitt **esp32**. Es öffnet sich ein Untermenü mit einer beeindruckenden Liste von Modellen.
4. Wählen Sie Ihr Modell:
    - Wählen Sie für ein Standardmodul mit 30 oder 38 Pins **DOIT ESP32 DEVKIT V1**.
    - Falls Sie eine neuere Version (S2, S3 oder C3) besitzen, suchen Sie nach dem entsprechenden Namen. (ex: **ESP32S3 Dev Module**).

Die Modelle sind nicht alphabetisch, sondern nach Marke oder Modell sortiert.

Alternativ können Sie auch den unten stehenden Auswahlfilter verwenden. *tools* :



Wählen Sie in der Dropdown-Liste aus *other board and port.* Im Pop-up-Fenster finden wir unsere Karte und den USB-Anschluss, an den sie angeschlossen ist:



Hier haben wir das **ESP32S3 Dev Module** und den **COM7**-Port ausgewählt. Der COM-Port kann bei Ihnen abweichen.

Auswahl einer bestimmten Karte

Bevor die Liste der Karten angezeigt werden kann, muss die IDE wissen, was ein ESP32 ist.

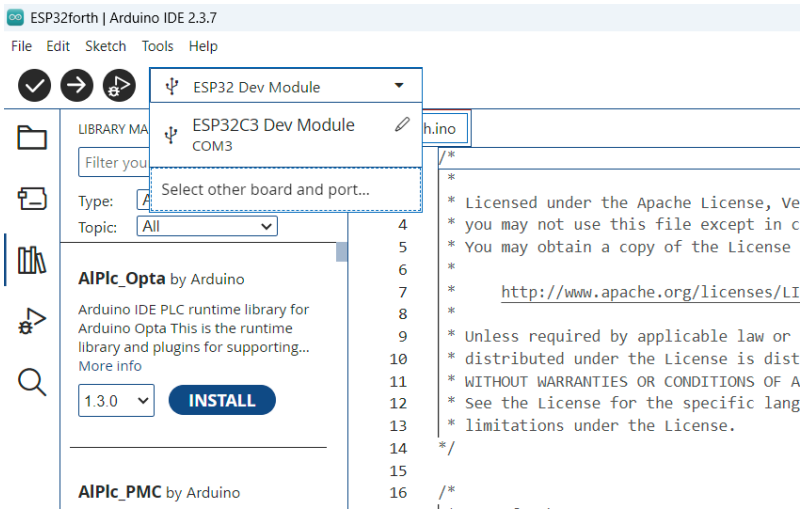
- Gehen Sie zu **File > Preferences**
- im Feld **Additionnal boards manager URLs** diesen Link ein: `https://dl.espressif.com/dl/package_esp32_index.json`
- Gehen Sie anschließend zu **Tools > Board > Boards Manager** , suchen Sie nach **ESP32** und klicken Sie auf **Instal**.

Sobald das Paket installiert ist, gehen Sie wie folgt vor, um Ihr Modul auszuwählen:

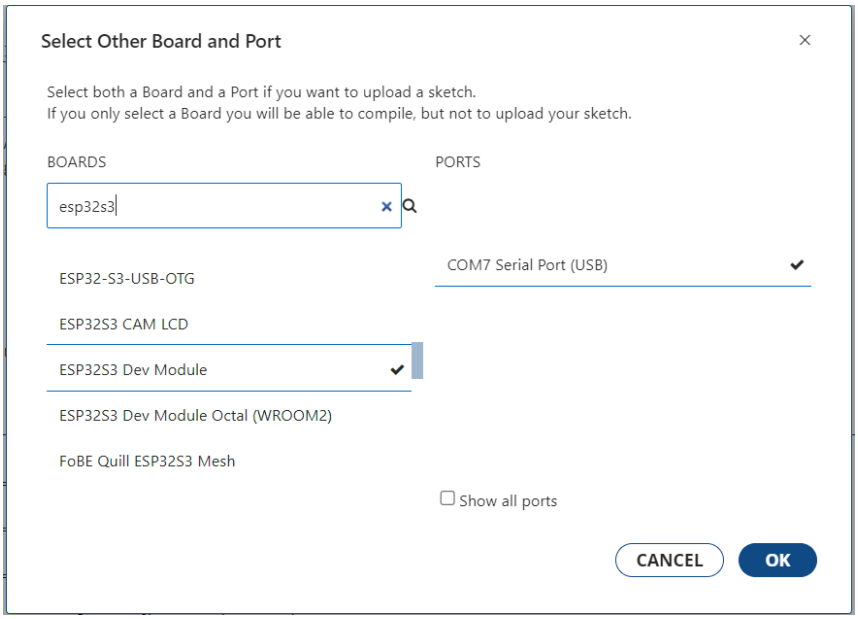
1. Öffnen Sie das Menü **Tools**.
2. Platzieren Sie Ihren Mauszeiger auf **Board : "..."** (standardmäßig oft "Arduino Uno").
3. Abschnitt **esp32**. Es öffnet sich ein Untermenü mit einer beeindruckenden Liste von Modellen.
4. Wählen Sie Ihr Modell:
    - Für ein Standardmodul mit 30 oder 38 Pins wählen Sie **DOIT ESP32 DEVKIT V1** .
    - Falls Sie eine neuere Version (S2, S3 oder C3) besitzen, suchen Sie nach dem entsprechenden Namen (z. B. **ESP32S3 Dev Module**).

Die Modelle sind nicht alphabetisch, sondern nach Marke oder Modell aufgelistet.

Die andere, einfachere Lösung besteht darin, diesen Selektor unterhalb von *Tools* zu verwenden :



Wählen Sie in der Dropdown-Liste *Select other board and port.* Suchen Sie im Popup-Fenster Ihre Platine und den USB-Anschluss, an den sie angeschlossen ist:



Hier haben wir das **ESP32S3-Entwicklungsmodul und den COM7-** Port ausgewählt . Der COM-Port kann bei Ihnen abweichen.

Die Werkzeuge

Die Werkzeuge können durch Klicken auf *Tools* in der Menüleiste aufgerufen werden. Es gibt viele dieser Werkzeuge:

```
Auto Format
Archive Sketch
Manage Libraries
Serial Monitor
Serial Plotter
--------------------------------
Firmware Updater
Upload SSL Root Certification
--------------------------------
Board:
Port:
Reload Bord Data
Get Board Info
--------------------------------
CPU Frequency:
Core Debug Level:
Erase All Flash Before Sketch Upload:
Events Run On:
Flash Frequency:
Flash Mode:
Flash Size:
JTAG Adapter:
Arduino Runs On:
Partition Scheme:
PSRAM:
Upload Speed:
ZigBee Mode:
--------------------------------
Programmer
Burn Bootloader
```

Unser Ziel ist es nicht, alle diese Werkzeuge im Detail zu beschreiben. Wir werden uns in erster Linie auf die für unser **ESP32forth -Projekt wesentlichen Parameter konzentrieren** .

Auto Format

Auto-Format -Funktion ist der beste Freund von Entwicklern, die Wert auf sauberes Arbeiten legen (oder auch von solchen, die etwas unordentlich sind).

In der Praxis dient es dazu, das Layout Ihres Codes automatisch so umzustrukturieren, dass er perfekt lesbar wird, ohne seine Funktionsweise zu verändern.

In der Programmierung ist Lesbarkeit von entscheidender Bedeutung. Auto Format berücksichtigt mehrere Aspekte:

5. **Einrückung** : Sie richtet linksbündige Elemente korrekt aus. Beispielsweise wird alles innerhalb einer `` `void loop() { ... }` `` -Funktion um ein Leerzeichen eingerückt, um die Hierarchie visuell darzustellen.

6. **Die geschweiften Klammern** : Sie ersetzen die { und } gemäß präzisen Programmierstandards und vermeiden so, sich in Kontrollstrukturen zu verheddern.
7. **Leerzeichen** : Er fügt Leerzeichen ein, wo sie fehlen (zum Beispiel um ein Gleichheitszeichen herum oder nach einem Komma), um den Text lesbarer zu machen.

Archive Sketch

Die Option *Archive Sketch* ist eine sehr nützliche, aber oft übersehene Funktion der Arduino IDE. Sie ermöglicht es Ihnen, sofort eine komprimierte Sicherungskopie Ihres aktuellen Projekts zu erstellen.

Bei der Arbeit an einem Elektronikprojekt fallen oft mehrere Dateien an (der Hauptcode, .h-Headerdateien, Unterdateien). Mit dieser Option können Sie alles übersichtlich „verpacken".

Was *Archive Sketch* tatsächlich leistet :

- **.zip-Komprimierung** : Dabei wird der gesamte Ordner Ihres Projekts in ein komprimiertes Archiv umgewandelt.
- **Automatische Zeitstempelung** : Die IDE benennt die Datei automatisch mit dem heutigen Datum (z. B. my_project_jan23a.zip). Dadurch können Sie Wiederherstellungspunkte erstellen, ohne Ihre vorherigen Versionen zu überschreiben.
- **Organisation** : Die komprimierte Datei befindet sich direkt in Ihrem Projektordner und kann direkt verschoben werden.

Der tatsächliche Nutzen:

- **Einfaches Teilen** : Wenn Sie Ihren Code an einen Freund oder Lehrer senden möchten, ist das Versenden einer einzelnen *.zip- Datei* viel einfacher als das Versenden eines ganzen Ordners mit mehreren Dateien.
- **Versionsverlauf** (Backup): Bevor Sie eine riskante Änderung vornehmen, die alles "zerstören" könnte, ermöglicht Ihnen eine schnelle Archivskizze, *eine* Kopie des aktuellen Arbeitszustands zu behalten.
- **Aufräumen** : Diese Funktion ermöglicht es Ihnen, Ihre alten Projekte kompakt auf Ihrem Computer oder in der Cloud zu speichern.

Manage Libraries

Die Option **Manage Libraries** ist eines der leistungsstärksten Werkzeuge der Arduino IDE. Sie ermöglicht es Ihnen, die Funktionen Ihres Boards zu erweitern, ohne das Rad neu erfinden zu müssen.

In der Informatik bezeichnet man als **Bibliothek** eine Sammlung von Code, die bereits von Fachleuten oder der Community geschrieben wurde, um eine bestimmte Komponente zum Laufen zu bringen (z. B. einen LCD-Bildschirm, einen Temperatursensor, einen Motor usw.).

Es handelt sich um eine Art kostenlosen "Anwendungsstore" für Ihren Code. Anstatt jeden Pin eines komplexen Bildschirms manuell zu konfigurieren, installieren Sie eine Bibliothek, die Ihnen einfache Befehle wie screen.write("Hello") bereitstellt.

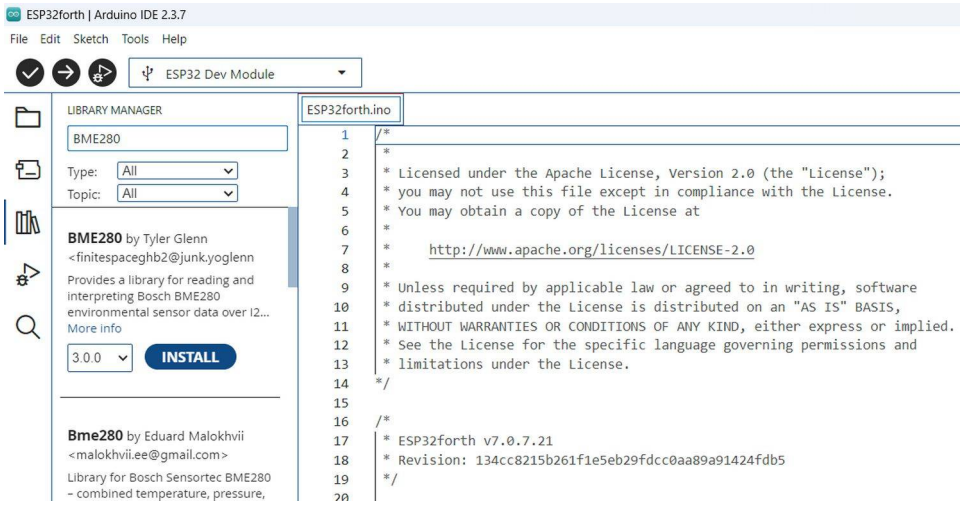
Der Manager wird für Folgendes verwendet:

- **Suchen Sie** nach Bibliotheken anhand des Namens oder der Funktionalität.
- Neue Bibliotheken mit einem Klick **installieren .**
- **Aktualisieren Sie** Ihre Bibliotheken, um Fehler zu beheben oder Funktionen hinzuzufügen.
- **Versionen verwalten** (falls ein Update Ihren Code beschädigt, können Sie auf eine ältere Version zurückgreifen).

Verwenden :

1. **Öffnen:** Gehen Sie zu **Outils > Manage Libraries** oder klicken Sie auf das Symbol "Bücher" in der linken Seitenleiste (in Version 2.x).
2. **Suche:** Geben Sie den Namen einer Komponente ein (z. B. „DHT11" für einen Feuchtigkeitssensor).
3. **Installation:** Klicken Sie auf die Schaltfläche „Installieren". Die IDE lädt alles herunter und installiert es am richtigen Ort.

Nehmen wir an, Sie kaufen einen **BME280-** Drucksensor und wissen wahrscheinlich nicht, wie Sie dessen elektrische Rohsignale interpretieren. Gehen Sie zu *Manage Libraries,* suchen Sie nach *BME280,* installieren Sie die Adafruit-Version, und plötzlich versteht Ihr Arduino den Befehl `sensor.readPressure()`.



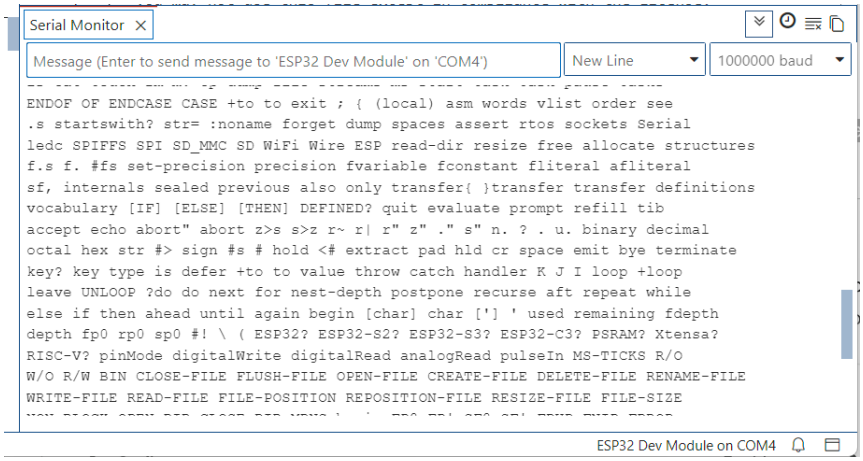
Wenn Sie im obigen Screenshot *auf INSTALL* klicken, wird die **BME280-** Bibliothek installiert. Solange Ihr Projekt jedoch nicht explizit auf die Funktionen dieser Bibliothek

zugreift, hat diese Installation keinen Einfluss auf die Größe des beim Kompilieren des Anwendungscodes generierten Binärcodes.

Serial Monitor

Der **Serial Monitor** ist das wichtigste Diagnosewerkzeug, das in die IDE integriert ist. Er ist der Kommunikationskanal, der es Ihrem Arduino-Board ermöglicht, in Echtzeit mit Ihrem Computer zu „sprechen".

Da ein ESP32-Board standardmäßig keinen Bildschirm besitzt, dient *Serial Monitor* als Einblick in die Vorgänge im Inneren des Mikrocontrollers:



Hier haben wir `page vlist` in diesem Terminalfenster ausgeführt.

Damit der Text korrekt angezeigt wird, muss die im seriellen Monitor eingestellte Übertragungsgeschwindigkeit (**Baudrate**) mit der in Ihrem Code angegebenen **übereinstimmen** (üblicherweise **9600** oder **115200**). Stimmen die Geschwindigkeiten nicht überein, werden seltsame oder unleserliche Zeichen („Kauderwelsch") angezeigt.

Board

Die **Board**- Option in der Arduino IDE ist die wichtigste Einstellung vor dem Hochladen Ihres Codes. Sie teilt der Software genau mit, welchen Hardwaretyp sie anspricht.

Da jeder Chip ein anderes Gehirn hat (Anzahl der Pins, Verarbeitungsgeschwindigkeit, Speicher), muss die IDE wissen, ob sie den Code für einen kleinen Arduino Uno oder für eine Hochleistungsmaschine wie den **ESP32** vorbereitet.

Bedeutung für den ESP32

Im Gegensatz zu Arduino, das seine eigenen Platinen herstellt, ist der **ESP32** ein Mikrocontroller, der von der Firma *Espressif* produziert wird , aber von Dutzenden verschiedener Hersteller verwendet wird (DOIT, Adafruit, LilyGO, Heltec...).

Jede Platine hat eine andere Pinbelegung. Wenn Sie die falsche „Board" wählen, riskieren Sie Folgendes:

- Der Code lässt sich überhaupt nicht hochladen.
- Dass Ihre Anschlüsse nicht übereinstimmen (z. B. möchten Sie Pin 2 einschalten, aber bei Ihrem Modell ist es Pin 4).

Beispiele gängiger ESP32-Boards

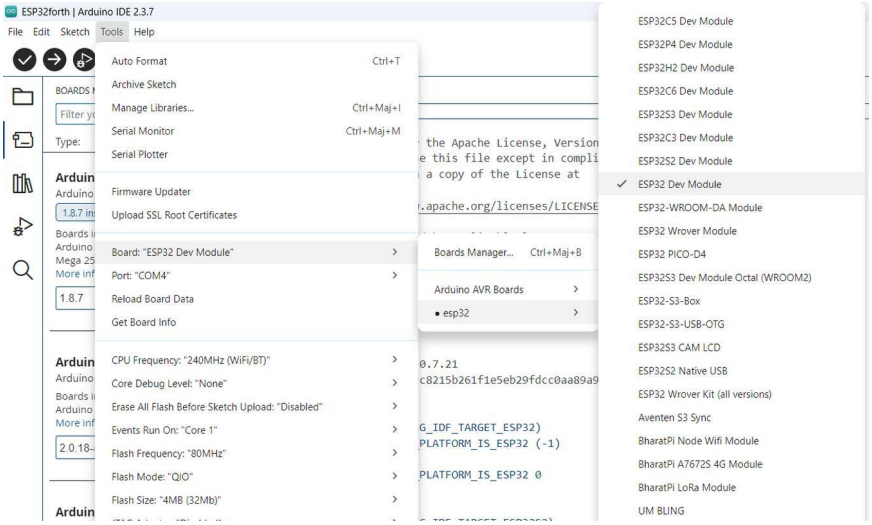
Unter *Tools → Board → ESP32* finden Sie eine umfangreiche Liste. Hier sind die gängigsten:

Name in der IDE	Typische Verwendung
DOIT ESP32 DEVKIT V1	Die klassischste und preisgünstigste Version (30 Pins). Ideal für Anfänger.
Adafruit Feather ESP32	Kompakte Größe mit integriertem Batterieanschluss.
ESP32-Entwicklungsmodul	Die „universelle“ Einstellung. Falls Sie Ihr genaues Modell nicht finden, funktioniert diese Einstellung oft standardmäßig.
Wemos D1 Mini ESP32	Eine Miniaturversion für Projekte mit begrenztem Platzangebot.

Konfiguration eines ESP32-Boards

Standardmäßig erkennt die Arduino IDE den ESP32 nicht. Er muss zuerst "angeleitet" werden:

1. **Hinzufügen der URL: Fügen Sie** in *Preferences* den Link zum Espressif-Kartenmanager hinzu.
2. **Installation: Suchen Sie** unter *Tools → Board → Boards Manager* nach "ESP32" und installieren Sie das Paket.
3. **Auswahl:** Nach der Installation gehen Sie zu *Tools → Board → ESP32* und wählen Sie Ihr spezifisches Modell aus (z. B. *DOIT ESP32 DEVKIT V1*).



Warnung: Nachdem Sie die Karte ausgewählt haben, überprüfen Sie auch den Port (COM) im Menü *Tools*, da Ihr Computer sonst nicht weiß, über welche „Leitung" der Code gesendet werden soll!

Port

Mit diesem Tool können Sie den seriellen Port auswählen, an den die zu programmierende ESP32-Platine angeschlossen ist.

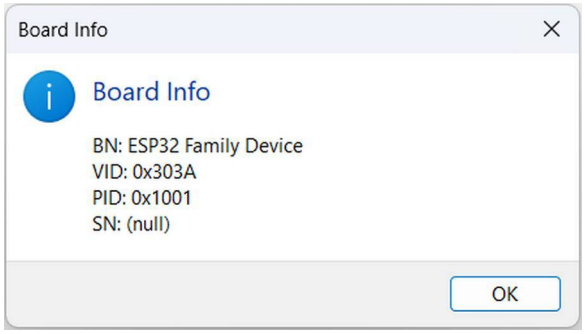
Die Arduino IDE zeigt nur aktive oder kürzlich verwendete Ports an. Wenn Ihr ESP32-Board an einen USB-Anschluss angeschlossen ist, erkennt die IDE das Board und zeigt den zugehörigen COM-Port an.

Get Board Info

Die Option **Get Board Info** ist eine Art „digitaler Ausweis", der die an Ihren USB-Anschluss angeschlossene Hardware abfragt.

Dies ist das perfekte Werkzeug, um zu überprüfen, ob Ihr Computer den ESP32 tatsächlich "sieht" und um genau zu ermitteln, welche elektronische Komponente für die Kommunikation verantwortlich ist.

Diese Option öffnet ein kleines Fenster, das die vom USB-Chip des Boards gesendeten technischen Rohdaten anzeigt. Dies ist nützlich für:



1. **Überprüfen Sie die Verbindung:** Wenn im Fenster eine Fehlermeldung angezeigt wird, bedeutet dies, dass das Kabel defekt ist oder die Treiber nicht installiert sind.
2. **Identifizierung eines Klons:** Wenn Sie ein „ESP32-kompatibles" Board erworben haben, zeigt Ihnen diese Option an,

welche Komponente für die Kommunikation verwendet wird (oft ein anderer Chip als im Original).

3. **Den richtigen Anschluss finden:** Wenn mehrere Geräte angeschlossen sind, bestätigt dies, dass Sie den richtigen COM-Anschluss ausgewählt haben.

Das Fenster zeigt üblicherweise drei Schlüsselcodes an:

- **BN (Board Name):** Der Name der Karte (sofern sie offiziell anerkannt ist).
- **VID (Vendor ID):** Die Kennung des Herstellers der USB-Komponente.
- **PID (Produkt-ID):** Die Kennung des Produktmodells.
- **SN (Serial Number):** Die eindeutige Seriennummer des Chips (sofern verfügbar).

Es kommt häufig vor, dass ein **ESP32** oder ein **generischer Arduino Nano** die Meldung **„Unknown Board"** anzeigt.

> **Warum?** Weil diese Boards oft generische USB-Chips (wie den **CH340** oder **CP2102**) verwenden. Die IDE erkennt zwar, dass sich am Ende des Kabels „etwas" befindet (die VID/PID wird angezeigt), kann aber nicht garantieren, dass es sich um ein offizielles Arduino-Board handelt.

Das ist in Ordnung: Solange eine VID- und PID-Nummer angezeigt wird, funktioniert die Kommunikation. Sie können die Meldung „Unbekanntes Board" ignorieren und Ihren Code wie gewohnt hochladen.

CPU Frequency

Die Option *CPU Frequency* bestimmt die „Reaktionsgeschwindigkeit" Ihres Mikrocontrollers. Sie ist der Herzschlag des Chips, reguliert durch einen Oszillator.

Je höher die Frequenz, desto mehr Anweisungen führt der ESP32 pro Sekunde aus.

Auf einem Standard-Arduino-Uno-Board ist die Frequenz fest eingestellt (16 MHz). Bei einem **ESP32** hingegen hat man oft die Wahl (üblicherweise zwischen **80 MHz, 160 MHz** und **240 MHz**). Die Änderung dieser Einstellung dient hauptsächlich zwei Zwecken:

- **Leistung** (Rechenleistung):
  Für Signalverarbeitung, Videodarstellung oder komplexe mathematische Berechnungen empfiehlt sich die maximale Frequenz (**240 MHz**). Der Prozessor verarbeitet Daten dadurch deutlich schneller.
- **Autonomie** (Energieeinsparung)
  Dies ist der häufigste Grund für die Reduzierung der Frequenz.
  - **Je höher die Frequenz** , desto mehr Strom verbraucht der Chip und desto mehr Wärme erzeugt er.
  - **Je niedriger die Frequenz** , desto mehr Batterieleistung wird gespart.

  Wenn Ihr Projekt lediglich die Temperatur einmal pro Minute misst, ist der Betrieb des Prozessors mit 240 MHz Energieverschwendung. Durch den Wechsel auf 80 MHz kann die Akkulaufzeit deutlich verlängert werden.

Beachten Sie, dass eine Frequenzänderung technisch gesehen nicht "kostenlos" ist:

- **WLAN und Bluetooth:** Auf dem ESP32 benötigen einige drahtlose Funktionen eine Mindestfrequenz (oft 80 MHz oder höher), um korrekt zu funktionieren. Bei zu niedriger Frequenz geht die WLAN-Verbindung verloren.
- **Timing:** Obwohl die IDE dies in den meisten Fällen übernimmt, können einige Timing-Berechnungen (wie z. B. sehr präzise Verzögerungen oder Kommunikationsprotokolle) gestört werden, wenn Sie die Frequenz mitten in einem Projekt ändern.

Zusammenfassung der typischen Einstellungen:

Frequenz	Ideale Verwendung
240 MHz	Aktives WLAN, Bluetooth, rechenintensive Anwendungen, Touchscreens.
160 MHz	Guter Kompromiss zwischen Leistung und Kraftstoffverbrauch.
80 MHz	Einfache Projekte, batteriebetriebene IoT-Sensoren, gelegentliches WLAN.
< 80 MHz	Sehr energiesparender Modus (Tiefschlaf), keine drahtlose Verbindung.

Core-Debug-Level

Die Option **Core Debug Level** ist eine Funktion, die speziell für leistungsstarke Boards wie den **ESP32** entwickelt wurde. Sie ermöglicht es Ihnen, den Umfang der technischen Details festzulegen, die das Board über seine internen Abläufe an den **Serial Monitor** sendet**.**

Das ist ein bisschen so, als würde man sein Auto bitten, entweder zu sagen: „Alles ist in Ordnung" oder jeden einzelnen Tropfen Benzin aufzulisten, der in den Motor eingespritzt wurde.

Der „Core" des ESP32 erledigt komplexe Aufgaben im Hintergrund (WLAN-Verbindung, Speicherverwaltung, Bluetooth). Falls Ihr Code ohne ersichtlichen Grund abstürzt, können Sie durch Erhöhen des Debug-Levels den genauen Fehler ermitteln, der aufgetreten ist.

Hier sind die verfügbaren Einstellungen (von am wenigsten bis am ausführlichsten):

- **None:** Dies ist die Standardeinstellung für die Produktion. Die Karte bleibt stumm, was Speicherplatz und Energie spart.
- **Error:** Die Karte meldet sich nur, wenn etwas Schwerwiegendes passiert (ein Absturz, ein defektes Bauteil).
- **Warn:** Zeigt Fehler an, aber auch Warnmeldungen über verdächtiges Verhalten, das noch nicht zum Systemabsturz geführt hat.

- **Info (Information):** Zeigt wichtige Schritte an, z. B. „WLAN-Verbindung erfolgreich" oder „IP-Adresse erhalten". Sehr nützlich zur Überwachung des normalen Ablaufs.
- **Debug:** Liefert technische Details zu internen Funktionen. Nützlich für fortgeschrittene Entwickler.
- **Verbose:** Die Karte protokolliert *jeden* Vorgang im Millisekundentakt. Achtung: Dies verlangsamt die Ausführung geringfügig und führt zu einer Überflutung des seriellen Monitors mit Text.

Wozu es verwenden?

1. **Fehlersuche bei Abstürzen:** Der ESP32 zeigt bei einem Absturz häufig die Fehlermeldung „Guru Meditation Error" an. Mit dem Debug-Level **Info** oder **Debug** erhalten Sie detailliertere Informationen zur Absturzursache (Speicher voll, Division durch Null usw.).
2. **WLAN-Überwachung:** Wenn Sie keine Verbindung zu Ihrer Box herstellen können, können Sie durch Einstellen des Überwachungslevels auf **„Info"** feststellen, ob es sich um ein Passwortproblem oder ein zu schwaches Signal handelt.

Praktischer Tipp: Lassen Sie diese Option in den meisten Fällen auf **None** oder **Error** eingestellt. Verwenden Sie **Info** oder **Verbose** nur, wenn Sie aktiv die Ursache eines bestimmten Fehlers untersuchen.

USB DFU On Boot

Unter *Tools* wird diese Auswahl für das ESP32S3 Dev Module Board angezeigt.

Diese Option mag etwas mysteriös erscheinen, ist aber für die Programmierung bestimmter moderner Boards (wie dem Arduino Nano ESP32 oder Boards, die auf dem ESP32-S2/S3 basieren) unerlässlich.

Was bedeutet **USB DFU On Boot** genau ? **DFU** steht für *Device Firmware Upgrade (Geräte-Firmware-Upgrade*). Es handelt sich um ein Standardprotokoll, das es ermöglicht, die Firmware eines Geräts über dessen USB-Anschluss neu zu flashen, ohne dass ein externer Programmierer oder ein komplexer Bootloader benötigt wird, der auf ein bestimmtes Signal wartet.

Um eine Karte in den Download-Modus zu versetzen, muss man normalerweise eine Taste (BOOT) gedrückt halten und gleichzeitig eine andere (RESET) drücken. Das ist eine ziemlich komplizierte Tastenkombination.

Durch Aktivieren **USB DFU On Boot** :

- **Enabled:** Beim Systemstart prüft die Karte zunächst, ob der Computer versucht, neuen Code über das DFU-Protokoll zu senden. Dies macht den Upload robuster, insbesondere wenn das vorherige Programm den seriellen Port zum Absturz gebracht hat.

- **Disabled:** Die Karte startet normal und führt Ihren Code sofort aus.

Wann sollte man es verwenden?

- **Aktivieren Sie diese Option,** wenn Sie Schwierigkeiten beim Hochladen Ihres Codes haben oder wenn Sie möchten, dass die Karte von Ihrem PC nativ als Update-Gerät erkannt wird, sobald sie angeschlossen ist.
- **Lassen Sie die Einstellung auf dem Standardwert** (oft deaktiviert), wenn alles einwandfrei funktioniert. Bei manchen Karten kann die Aktivierung den Programmstart geringfügig verlangsamen, da die Karte kurz wartet, bis neuer Code eintrifft.

WICHTIG: Im Fall des Arduino Nano ESP32: Bei diesem speziellen Board ist die Option oft standardmäßig *aktiviert* , damit die IDE die Kontrolle über das Board problemlos wiedererlangen kann, selbst wenn der Benutzercode die normale serielle Kommunikation blockiert.

Zustand	Startup-Verhalten	Hauptverwendung
Ermöglicht	Die Karte stellt eine DFU-Schnittstelle über USB bereit.	Kartenwiederherstellung, Upload schwierig.
Deaktiviert	Die Karte gibt sofort den Benutzercode ein.	Normaler Gebrauch, schneller Start.

Erase All Flash Before Sketch Upload

Option **Erase All Flash Before Sketch Upload** ist ein Befehl zur „tiefgreifenden Bereinigung" Ihres ESP32.

Standardmäßig ersetzt die IDE beim Hochladen eines neuen Programms (Sketch) auf Ihre Speicherkarte nur den für den Code benötigten Speicherbereich. Der restliche Flash-Speicher (in dem Ihre WLAN-Einstellungen, SPIFFS-/LittleFS-Dateien oder gespeicherte Daten abgelegt sind) bleibt unberührt.

Durch Aktivieren dieser Option wird die IDE angewiesen, den Chip vor dem Schreiben neuen Codes **vollständig zu formatieren .**

Es ist weder notwendig noch empfehlenswert, diese Funktion jedes Mal zu verwenden, da dies den Upload-Vorgang verlangsamt. In folgenden Fällen ist sie jedoch unerlässlich:

- **Unvorhersehbares Verhalten:** Ihr Code ist korrekt, aber die Karte verhält sich seltsam, startet in einer Schleife neu oder merkt sich alte WLAN-Einstellungen, die Sie nicht mehr benötigen.
- **Dateisystemänderung:** Wenn Sie von einem Projekt, das **SPIFFS** verwendet, zu einem Projekt wechseln, das **LittleFS** verwendet , ist es am besten, den gesamten Flash-Speicher zu löschen, um Partitionskonflikte zu vermeiden.
- **Nach einem schweren Absturz:** Wenn der Speicher aufgrund eines Fehlers in der Speicherverwaltung beschädigt wurde, stellt ein vollständiges Löschen die Karte in ihren „Werkszustand" zurück.

- **Verkauf oder Spende der Karte:** Um sicherzustellen, dass keine sensiblen Daten (WLAN-Passwort, API-Schlüssel) im Speicher verbleiben.

Warnung: Wie der Name schon sagt, ist dieser Vorgang unumkehrbar.

- Alle Ihre **gespeicherten Einstellungen** (über die Bibliothek `Preferences.h`) werden gelöscht.
- Alle Ihre **gespeicherten Dateien** (Bilder, Webseiten, Protokolle) werden gelöscht.
- Die Uploadzeit wird etwas länger sein, da die IDE vor dem Schreiben einen vollständigen Löschbefehl (`chip_erase`) senden muss.

Wie aktiviere ich es?

1. Gehen Sie zum Menü **Tools**.
2. Suchen Sie nach der Zeile „**Erase All Flash Before Sketch Upload**.
3. Wählen Sie **Enabled**.
4. **Wichtig:** Sobald Ihr Problem gelöst ist, stellen Sie diese Option wieder auf **Disabled**, um bei Ihren nächsten Versuchen Zeit zu sparen.

Events Run On

Die Option **Events Run On** ist eine erweiterte Einstellung, die speziell für die **ESP32** -Architektur entwickelt wurde, welche die Besonderheit besitzt, **zwei Kerne** (Kern 0 und Kern 1) anstelle von nur einem zu besitzen.

Mit dieser Option können Sie auswählen, auf welchem "Gehirn" das System **die Systemereignisse verarbeiten** soll (hauptsächlich WLAN-, Bluetooth- und TCP/IP-Netzwerkaufgaben).

Warum die Aufgaben trennen?

Standardmäßig verteilt der ESP32 seine Arbeitslast wie folgt:

- **Core 1:** Hier wird Ihr Haupt-Arduino-Code ausgeführt (die `setup()`- und `loop()` -Funktionen).
- **Core 0:** Hier übernimmt im Allgemeinen das "Betriebssystem" (FreeRTOS) die komplexe drahtlose Kommunikation.

Option **Events Run On** können Sie die Ereignisbehandlung auf dem einen oder anderen System erzwingen.

Die verfügbaren Optionen:

1. **Kern 0 (Standard/Empfohlen):** Das System verwaltet WLAN und Bluetooth auf Kern 0. Dadurch steht Kern 1 vollständig zur Verfügung, um Ihren Code ohne Unterbrechung auszuführen. Dies ist die stabilste Konfiguration für die meisten Projekte.

2. **Kern 1:** Das System verarbeitet alle Vorgänge auf demselben Kern wie Ihr Code. Dies kann in bestimmten Synchronisierungsfällen nützlich sein, birgt aber Risiken: Führt Ihr Code eine Berechnung durch, die zu lange dauert (z. B. eine große `delay() -Schleife`), kann dies die WLAN-Verbindung unterbrechen und zu einer Verbindungsunterbrechung führen.

Sie werden diese Einstellung wahrscheinlich nie ändern müssen, es sei denn:

- **Sie entwickeln eine kritische Echtzeitanwendung:** Wenn Sie absolute Präzision auf Kern 1 benötigen (z. B. um einen Motor sehr genau zu steuern), ist es unbedingt erforderlich, dass WiFi-Ereignisse (Kern 0) Ihre Zeitsteuerung nicht stören.
- **Latenzoptimierung:** In hochspezialisierten Netzwerkprojekten kann die Verlagerung der Ereignisverarbeitung die Reaktionszeit manchmal um einige Mikrosekunden reduzieren.

Wichtiger Hinweis: Sollten Sie diese Einstellung ändern und Ihr ESP32 daraufhin ständig neu starten oder die WLAN-Verbindung instabil werden, stellen Sie die Standardeinstellung (**Core 0**) sofort wieder her.

Flash Frequency

Die Option **Flash-Frequenz** definiert die Kommunikationsgeschwindigkeit zwischen dem ESP32-Prozessor und seinem externen Flash-Speicherchip.

Anders als bei einem Computer, bei dem Arbeitsspeicher und Speicher getrennt sind, lädt der ESP32 die Programmanweisungen direkt aus seinem Flash-Chip zur Ausführung. Die Geschwindigkeit dieser Datenübertragung beeinflusst unmittelbar die Gesamtausführungsgeschwindigkeit Ihres Codes.

In der Arduino IDE haben Sie im Allgemeinen die Wahl zwischen mehreren Frequenzen:

- **40 MHz:** Dies ist die sicherste und stabilste Einstellung. Es ist die Standardgeschwindigkeit, die mit allen auf dem Markt erhältlichen ESP32-Modulen funktioniert.
- **80 MHz:** Dies ist die Einstellung für optimale Leistung. Die Kommunikation ist doppelt so schnell. Die meisten modernen Module (ESP32-WROOM usw.) unterstützen diese Frequenz sehr gut.

Leistungssteigerung (80 MHz)

Durch den Wechsel auf 80 MHz wird Ihr Programm schneller in den Speicher geladen. Bei sehr großem Code oder der Verarbeitung vieler im Flash-Speicher abgelegter Dateien (z. B. Bildschirmabbilder, Webserver-Dateien) verbessert sich die Reaktionsfähigkeit.

Stabilität und Zuverlässigkeit (40 MHz)

Wenn Ihre Karte sich plötzlich ungewöhnlich verhält:

- Der Code stürzt willkürlich ab.
- Im seriellen Monitor erscheint die Fehlermeldung „`Flash read err“` .

- Der Upload schlägt regelmäßig fehl. Stellen Sie daher wieder auf **40 MHz** um**.** Einige minderwertige Flash-Chips oder übermäßig lange Kabel an selbstgebauten Modulen unterstützen keine hohen Geschwindigkeiten.

Geschwindigkeit ist nur ein Teil der Gleichung. Sie arbeitet oft zusammen mit **dem Blitzmodus** (wie QIO oder DIO).

- **40 MHz + DIO:** Maximale Sicherheit.
- **80 MHz + QIO:** Maximale Leistung (doppelte Geschwindigkeit und 4 Datenleitungen statt 2).

Praktischer Tipp: Für die meisten Projekte können Sie diese Einstellung bei **80 MHz** belassen. Sollten Sie feststellen, dass Ihr ESP32 überhitzt oder sich ohne ersichtlichen Grund neu startet, versuchen Sie, die Taktfrequenz auf **40 MHz** zu reduzieren, um die Ursache des Problems zu finden.

Flash Mode

Der **Flash Mode** definiert, wie der Prozessor des ESP32 mit seinem Flash-Speicherchip (in dem Ihr Programm gespeichert ist) kommuniziert. Es handelt sich um eine Bandbreiteneinstellung: Sie bestimmt, wie viele Datenleitungen gleichzeitig zum Lesen von Anweisungen verwendet werden.

Das ist ein bisschen so, als würde man eine Straße mit einer oder mehreren Fahrspuren vergleichen: Je mehr Fahrspuren es gibt, desto schneller fließen die Daten.

Im Menü der Arduino IDE finden Sie in der Regel diese vier Optionen:

QIO (Quad I/O) – *Schnellste*

- **Funktionsweise:** Verwendet **4** Datenleitungen für Adressen und Daten.
- **Leistung:** Dies ist der Modus mit der höchsten Leistung.
- **Kompatibilität:** Der Flash-Chip und die Verkabelung müssen den Quad-Modus unterstützen. Falls Ihr ESP32 nach dem Hochladen in diesem Modus nicht startet, ist Ihre Hardware nicht kompatibel.

QOUT (Quad-Ausgang)

- **Funktionsweise:** Verwendet **4 Drähte** ausschließlich für die Datenübertragung (Adressen werden über einen einzigen Draht übertragen).
- **Leistung:** Etwas langsamer als QIO, aber auf manchen Chips stabiler.

DIO (Dual I/O) – *Der Standard*

- **Funktionsweise:** Verwendet **2** Datenleitungen für Adressen und Daten.
- **Leistung:** Durchschnittsgeschwindigkeit.
- **Anwendung:** Dies ist der Standardmodus für viele ESP32-Module, da er sehr zuverlässig ist.

DOUT (Dual Output) – *Die kompatibelste Lösung*

- **Funktionsweise:** Verwendet nur **2 Drähte** für die Datenübertragung.
- **Leistung:** Die langsamste der vier.
- **Anwendung:** Nur als letzte Möglichkeit verwenden, wenn keine andere Methode funktioniert. Es ist mit nahezu allen auf dem Markt erhältlichen Flash-Chips kompatibel.

Die Wahl hängt vom genauen Modell Ihres ESP32-Moduls ab (dem kleinen Metallquadrat auf Ihrer Platine):

- **Für einen Standard-ESP32-WROOM** oder **WROVER** ist der **DIO-** Modus die sicherste Wahl. Viele unterstützen zwar **QIO** , der Geschwindigkeitsgewinn ist aber bei einfachen Programmen nicht immer spürbar.
- **Wenn Sie hohe Leistung benötigen (Grafikdarstellung, Audio):** Versuchen Sie den **QIO-** Modus . Falls die Karte in einer Endlosschleife mit der Fehlermeldung „Flash read err" neu startet, wechseln Sie zurück in den **DIO**-Modus .
- **Achten Sie auf die verwendeten Pins:** Der Quad-Modus (QIO/QOUT) verwendet bestimmte Pins (GPIO 6 bis 11). Wenn Sie versuchen, diese Pins in Ihrem Projekt für etwas anderes zu verwenden, verursacht der Quad-Modus einen Konflikt und Ihr Programm stürzt ab.

Zusammenfassend:

Mode	Anzahl der Datenthreads	Relativgeschwindigkeit	Stabilität
QIO	4	⭐⭐⭐⭐	Durchschnitt
QOUT	4	⭐⭐⭐	Gut
DIO	2	⭐⭐	Exzellent
ZWEIFELN	2	⭐	Maximal

Flash Size

Die Option **Flash Size** teilt der Arduino IDE die gesamte Speicherkapazität des Speicherchips auf Ihrem ESP32-Modul mit.

Im Gegensatz zu einem Computer, bei dem das System die Größe der Festplatte automatisch erkennt, muss hier der Compiler diese genaue Grenze kennen, um den Speicherplatz zu organisieren (die „Partitionierung"), bevor er Ihr Programm sendet.

Der Flash-Speicher des ESP32 enthält alles, was zum Überstehen eines Stromausfalls notwendig ist:

1. **Ihr Programm** (der kompilierte Code).
2. **Das Betriebssystem** (FreeRTOS).
3. **Benutzerdaten** (LittleFS/SPIFFS-Dateien, WLAN-Einstellungen).

4. **Die OTA-** Zone (Over-The-Air): ein reservierter Bereich für drahtlose Kartenaktualisierungen.

Wenn Sie die IDE auf **2 MB** einstellen, obwohl Ihre Karte **4 MB** hat, nutzen Sie nur die Hälfte der verfügbaren Kapazität. Umgekehrt: Wenn Sie sie auf **4 MB** einstellen, obwohl Ihre Karte nur **2 MB** hat, schlägt der Upload fehl oder die Karte stürzt ab, sobald sie versucht, auf einen nicht vorhandenen Speicherbereich zuzugreifen.

Gängige Größen für den ESP32

- **4 MB (4 Megabyte):** Dies ist der absolute Standard. Die überwiegende Mehrheit der handelsüblichen **ESP32-WROOM-32- Module verfügt über 4 MB.**
- **2 MB:** Häufig bei Miniatur- oder sehr günstigen Versionen zu finden.
- **8 MB / 16 MB:** Werden auf High-End-Karten (wie einigen M5Stack-Modellen oder ESP32-S3) verwendet, die für die Speicherung großer Mengen an Bildern oder Tönen ausgelegt sind.

Wie finde ich die Größe meiner Karte heraus?

Wenn Sie die Kapazität Ihrer Karte nicht kennen, können Sie die Option **„Get Board Info"** verwenden, die wir bereits erwähnt haben. Diese gibt jedoch nicht immer die genaue Größe des Flash-Speichers an.

Die zuverlässigste Methode ist, beim Start oder während des Uploads den seriellen Monitor zu überprüfen. Das von der IDE verwendete Tool `esptool` zeigt oft eine Zeile wie diese an:

```
Detected Flash size: 4MB
```

Die Größe des Flash-Speichers ist untrennbar mit dem **Partition Scheme**. Beispielsweise kann man bei einer **4-MB-** Karte den Speicherplatz wie folgt aufteilen:

- **Default:** ~1,2 MB für den Code, 1,5 MB für die Dateien, der Rest für Updates.
- **No OTA (Große APP):** ~3 MB für den Code (wenn Ihr Programm sehr groß ist), aber Sie verlieren die Möglichkeit, die Karte über WLAN zu aktualisieren.

Tipp: Wenn Sie sich nicht sicher sind, lassen Sie diese Einstellung bei **4 MB** . Dies ist die „universelle" Einstellung für 90 % der ESP32-Entwicklungsboards (DevKit V1, NodeMCU-32S usw.).

JTAG-Adapter

JTAG-Adapter- Option in der Arduino IDE für ESP32 ist eine hochmoderne Einstellung, mit der Sie das für das **Echtzeit-Debugging** verwendete Hardware-Tool auswählen können.

Während der *serielle Monitor* lediglich die Anzeige von Text ermöglicht, um zu verstehen, was passiert, erlaubt **JTAG** das "Einfrieren" des Prozessors, die Untersuchung jeder einzelnen Variable und das Durchgehen des Codes Zeile für Zeile während der Ausführung.

JTAG (*Joint Test Action Group*) ist eine standardisierte physikalische Schnittstelle zum Testen und Debuggen von Mikrocontrollern. Bei einem ESP32 ist dafür typischerweise ein externes Gerät (der Adapter) erforderlich, das zwischen Computer und bestimmten Pins auf der Platine angeschlossen wird.

Welchen Zweck hat diese Option in der IDE?

In der Arduino IDE 2.x gibt es eine Schaltfläche „Debugging starten" (das Insektensymbol mit dem Wiedergabepfeil). Damit diese Schaltfläche funktioniert, muss die IDE wissen, welche Hardware die Verbindung zwischen Ihrem PC und dem Chip herstellt.

Hier sind die gängigen Gerichte, die Sie im Menü finden:

1. **Integrated JTAG (ESP32-S3/C3):** Neuere Chips wie der ESP32-S3 verfügen über einen integrierten JTAG-Konverter. Sie benötigen keine zusätzliche Hardware, lediglich ein USB-Kabel.
2. **ESP-Prog:** Dies ist das offizielle Tool von Espressif. Es handelt sich um eine kleine Leiterplatte, die an die GPIO-Pins des ESP32 angeschlossen wird.
3. **J-Link / FT2232:** Sehr schnelle universelle Profi-Adapter.

Wozu es verwenden?

- **Haltepunkte:** Sie können dem ESP32 mitteilen: „Stopp, sobald Sie Zeile 42 erreichen". Anschließend können Sie überprüfen, ob Ihre Variablen die korrekten Werte haben.
- **Crash-Analyse:** Wenn Ihre Karte ständig neu startet, ermöglicht Ihnen JTAG, genau zu sehen, welche Anweisung den Systemfehler verursacht hat.
- **Geschwindigkeit:** Bei sehr großen Projekten ermöglicht JTAG das schnellere Hochladen von Code als über die klassische serielle Schnittstelle.

Warum nicht immer nutzen?

- **Komplexität:** Dies erfordert die Konfiguration spezifischer Treiber (oft mit einem Hilfsprogramm wie *Zadig*).
- **Belegte Pins:** JTAG verwendet mehrere GPIO-Pins (häufig 12, 13, 14 und 15). Wenn Sie diese Pins für Sensoren oder Displays verwenden, können Sie JTAG nicht gleichzeitig nutzen.

Tipp: Anfängern ohne ESP-Prog-Box oder S-Series-Chip empfehlen wir, die Standardeinstellung beizubehalten. Das Debuggen mit `Serial.print()` ist weiterhin die einfachste und am weitesten verbreitete Methode.

Arduino Runs On

Option **Arduino Runs On** ist der Option *Events Run On,* die wir zuvor kennengelernt haben, sehr ähnlich, aber diesmal betrifft sie **Ihren eigenen Code** (den „Sketch").

Da der ESP32 über zwei Kerne verfügt (Kern 0 und Kern 1), können Sie mit dieser Option festlegen, welcher Kern für die Ausführung Ihrer `setup()`- und `loop()`- Funktionen zuständig sein soll.

Bei einem Standard-ESP32 haben Sie die Wahl zwischen:

1. **Kern 1 (Standard / Empfohlen):** Dies ist die Standardeinstellung. Ihr Code läuft auf Kern 1, während Kern 0 für wichtige Systemaufgaben (WLAN, Bluetooth, TCP/IP-Stack) reserviert ist.

    - **Vorteil:** Ihr Programm verlangsamt die drahtlose Verbindung nicht und umgekehrt.

2. **Core 0:** Ihr Code wird auf denselben Kern wie die Netzwerkfunktionen verschoben.
    - **Anwendung:** Diese Einstellung wird sehr selten verwendet. Sie wird gelegentlich vorgenommen, um Core 1 vollständig freizugeben und so manuell ultraprioritäre, auf die Mikrosekunde genau getaktete Aufgaben über FreeRTOS (das interne Betriebssystem) zu starten.

Der ESP32 verwendet ein Echtzeitbetriebssystem namens **FreeRTOS** . Selbst wenn Sie das Gefühl haben, einfachen Code zu schreiben, erstellt die IDE im Hintergrund eine „Task", die `loop()` enthält.

- **Wenn Sie Core 1 wählen:** Falls Ihr Code "abstürzt" oder stoppt (zum Beispiel durch eine schlecht programmierte Endlosschleife), kann das WLAN auf Core 0 noch einige Augenblicke weiter funktionieren, bevor der Watchdog das Problem *bemerkt .*
- **Bei Auswahl von Kern 0:** Ist Ihr Code zu umfangreich oder enthält er übermäßig lange `delay()-Funktionen` , wird der Prozessor überlastet. Die WLAN-Verbindung kann dann nicht mehr aufrechterhalten werden, und Ihre Netzwerkkarte trennt die Verbindung ständig.

Zusammenfassend:

Option	Ziel Ihres Codes	Verwenden
Kern 1	Anwendungskern	**In 99 % der Fälle** ist dies die stabile Konfiguration.
Kern 0	Kernsystem	Sehr fortgeschrittene Projekte, bei denen es darum geht, die Rollen umzukehren.

Partition Scheme

Die Option **Partition Scheme** ist eine der wichtigsten Einstellungen für den ESP32. Sie definiert, wie der gesamte Flash-Speicher auf Ihrer Karte (z. B. 4 MB) in virtuelle „Schubladen" aufgeteilt wird.

Da der ESP32 Ihren Code, Systemdaten und manchmal auch Dateien (Bilder, Webseiten) speichern muss, müssen Sie ihm im Voraus mitteilen, welche Größe für jeden Abschnitt reserviert werden soll.

Standardmäßig ist der Speicherplatz so aufgeteilt, dass drahtlose Updates möglich sind.(**OTA)** aufgeteilt. Das bedeutet, dass die IDE zwei identische Bereiche für Ihr Programm reserviert: einen für den aktuellen Code und einen leeren Bereich zum Empfangen des Updates.

Daher stehen Ihnen von 4 MB nur etwa 1,2 MB für Ihren eigentlichen Code zur Verfügung. Überschreitet Ihr Programm diese Größe, erhalten Sie die Fehlermeldung *„Sketch too big* . An dieser Stelle kommt die Partitionsänderung ins Spiel.

Die gängigsten Optionen

Hier sind die Einstellungen, die Sie normalerweise im Menü **Tools > Partition Scheme** finden :

- **Standard (4 MB mit OTA):**
  - Code: ~1,2 MB
  - Dateien (SPIFFS/LittleFS): ~1,5 MB
  - *Einsatzgebiet:* Der Standard für klassische Projekte mit WLAN-Aufrüstung.
- **Nein. OTA (2 MB App / 2 MB SPIFFS):**
  - Code: 2 MB
  - Dateien: 2 MB
  - *Einsatzgebiet:* Für Projekte mit vielen Bildern oder Daten, aber ohne drahtlose Aktualisierung.
- **Riesige App (3 MB, kein OTA):**
  - Code: **3 MB**
  - Dateien: Sehr wenige (ca. 190 KB).
  - *Einsatzgebiet:* Unverzichtbar für große Programme (einfache künstliche Intelligenz, rechenintensive Bluetooth-Anwendungen oder komplexe grafische Benutzeroberflächen).
- **Minimal SPIFFS:**
  - Dadurch wird fast der gesamte Speicherplatz für den Code reserviert und der Dateispeicherplatz auf ein absolutes Minimum reduziert.

Welche Auswirkungen hat das auf Ihren Code?

Die Wahl der Partition verändert die Speicherstruktur **während des Uploads** .

Warnung: Wenn Sie Ihr Partitionsschema ändern, ändert sich der Speicherort Ihrer Daten. Dies kann dazu führen, dass in SPIFFS oder LittleFS gespeicherte Dateien gelöscht werden. Daher wird empfohlen, das Schema zu Beginn Ihres Projekts festzulegen und es anschließend nicht mehr zu ändern.

Wenn Sie ESP32forth mit vielen Optionen verwenden, wählen Sie die zweite Option:

Werkzeuge → **Partitionsschema** :

- "Kein OTA (2 MB App / 2 MB SPIFFS)" = 2 MB für Ihren Code
- " **Riesige App (3 MB, kein OTA)** " = 3 MB für Ihren ESP32forth-Code

Auch wenn dadurch der FLASH-Speicherplatz für SPIFFS reduziert wird, bleibt genügend Platz für den FORTH-Quellcode.

Wie soll man sich entscheiden?

1. **Ist Ihr Code zu groß?** Wechseln Sie zu **Huge APP**.
2. **Müssen Sie viele Dateien (Sounds, HTML) speichern?** Wechseln Sie zu **„No OTA (Large SPIFFS)"** .
3. **Möchten Sie Ihre Karte über WLAN aktualisieren können?** Bleiben Sie im Modus **„ with OTA"** .

Zusammenstellungsbericht anzeigen

Eine gute Angewohnheit: Die Größe Ihres Programms im Auge zu behalten, hilft, viel Frust beim Hochladen zu vermeiden.

Arduino IDE lesen und interpretieren .

Jedes Mal, wenn Sie auf **„Prüfen"** (das Häkchen-Symbol) oder **„Hochladen" klicken** , zeigt die IDE eine Zusammenfassung in der schwarzen Konsole am unteren Rand des Fensters an.

So sieht es aus:

```
The sketch uses 262,144 bytes (20%) of the program storage space. The maximum is
1,310,720 bytes. Global variables use 15,320 bytes (4%) of the dynamic memory...
```

- **Speicherplatz (Flash):** Dies ist die Größe Ihres Code-„Speicherbereichs", wie sie durch Ihr **Partition Scheme** definiert ist. Wenn Sie 90–95 % erreicht haben, ist es Zeit, auf eine größere Partition (z. B. *Huge APP)* umzusteigen .
- **Dynamischer Speicher (RAM):** Dies ist der Arbeitsspeicher. Ist dieser zu hoch (nahe 100 %), kann Ihr ESP32 während des Betriebs einfrieren oder unerwartet neu starten.

PSRAM

PSRAM (Pseudo-Static Random Access Memory) ist eine Erweiterung des Arbeitsspeichers (RAM) des ESP32 **.**

Um den Nutzen zu verstehen, stellen Sie sich den internen RAM des ESP32 als kleinen, sehr schnellen Arbeitsbereich vor. Bei großen Projekten (Bildverarbeitung, Audio, künstliche Intelligenz) ist dieser Arbeitsbereich jedoch zu klein. Der PSRAM ist wie ein **großer Architektentisch daneben** : Er ist etwas langsamer als der Hauptarbeitsbereich, bietet aber enorm viel zusätzlichen Speicherplatz.

Der Standard-ESP32 verfügt über ca. **520 KB** internen RAM. Das ist viel für einfache Elektronik, aber sehr wenig für:

- **Video/Foto:** Speichert ein einzelnes hochauflösendes Bild von einer Kamera (ESP32-CAM).
- **Audio:** Erstellen Sie einen Puffer, um Streaming-Musik ohne Unterbrechung abzuspielen.
- **Anzeige:** Verwaltung von Farb-Touchscreens mit flüssigen grafischen Benutzeroberflächen (LVGL).
- **Das Web:** Betrieb eines komplexen Servers mit vielen dynamischen Daten.

Mit PSRAM können Sie den verfügbaren Speicher von 520 KB auf **2 MB, 4 MB oder sogar 8 MB erweitern.**

Wenn Ihre Platine über einen PSRAM-Chip verfügt (wie die Modelle **WROVER** oder **ESP32-S3-WROOM-1**), ist dieser nicht unbedingt standardmäßig aktiv.

1. Gehen Sie zum Menü **„Tools"** .
2. Halten Sie Ausschau nach der **PSRAM-** Option .
3. Wählen Sie **Aktiviert** (oder "OPI PSRAM" / "QSPI PSRAM", je nach Modell).

Grenzen, die zu beachten sind

- **Geschwindigkeit:** PSRAM ist über einen seriellen Bus (SPI) angebunden. Daher ist es langsamer als interner RAM. Es dient der Speicherung großer Datenmengen, jedoch nicht für ultraschnelle mathematische Berechnungen.
- **Verbrauch:** Es verbraucht etwas mehr Energie, die im Batteriebetrieb überwacht werden sollte.
- **Pin-Konflikt:** Bei einigen älteren ESP32-Modellen belegt die Verwendung von PSRAM einige GPIO-Pins (häufig 16 und 17). Überprüfen Sie Ihre Pinbelegung sorgfältig.

Wie erkennt man, ob man PSRAM hat? Schauen Sie sich das Etikett auf dem Metallmodul Ihrer Karte an. Steht dort **WROVER** , verfügt Ihre Karte über PSRAM. Steht dort **WROOM** , ist dies in der Regel nicht der Fall (außer bei speziellen Versionen).

Upload Speed

Die Option **Upload Speed** legt fest, mit welcher Geschwindigkeit die Arduino IDE Ihr kompiliertes Programm über das USB-Kabel an das ESP32-Board sendet.

Baud (Bits pro Sekunde) gemessen . Sie entspricht der Datenübertragungsrate zwischen Ihrem Computer und Ihrer Karte während der Programmierphase.

Im Gegensatz zu Arduino Uno-Boards, die auf 115.200 Baud beschränkt sind, ist der ESP32 ein echter Geschwindigkeitsfresser. Sie finden in der Regel folgende Optionen:

- **115.200:** Die „sichere" Geschwindigkeit. Sie ist langsam, aber sehr zuverlässig.
- **460 800:** Ein ausgezeichneter Kompromiss, sehr weit verbreitet.
- **921.600:** Die aktuelle Höchstgeschwindigkeit. Das ist fast 10-mal schneller als beim Standard-Arduino.

Warum sollte man sich für die eine Geschwindigkeit und gegen die andere entscheiden?

Die Zeitersparnis : Wenn Sie ein großes Projekt entwickeln (mit vielen Bibliotheken wie WiFi oder Bluetooth), kann Ihre Binärdatei mehrere Megabyte groß sein.

- Bei **115.200** kann der Upload 30 bis 40 Sekunden dauern.
- Bei **921.600** Tests dauert es nur 5 bis 10 Sekunden. Im Laufe eines Arbeitstages mit Dutzenden von Tests ist der Unterschied enorm.

Zuverlässigkeit (Stabilität) : Wenn Sie die Geschwindigkeit zu hoch einstellen, riskieren Sie Übertragungsfehler (`Paketinhaltsabweichungen` oder `Verbindungsfehler`). Dies geschieht üblicherweise aus folgenden Gründen:

- **Ein minderwertiges USB-Kabel:** Zu lang oder schlecht abgeschirmt.
- **Ein günstiger USB-zu-Seriell-Konverter:** Einige „Klon"-Karten verwenden Chips, die hohe Geschwindigkeiten nicht gut unterstützen.
- **Störungen:** Wenn sich in Ihrer Umgebung laute Motoren oder Netzteile befinden.

Ändert dies die Geschwindigkeit meines Programms?

Nein. Das ist ein weit verbreiteter Irrtum.

- **Upload-Geschwindigkeit:** Betrifft nur die Übertragung des Codes (die Programmierphase).
- **Baudrate (Serial.begin):** Dies ist die Kommunikationsgeschwindigkeit während der Programmausführung (für den seriellen Monitor).

Sie können problemlos Daten auf **921 600 hochladen und mit dem seriellen Monitor auf 115 200** kommunizieren.

USB CDC On Boot

Die Option **„USB CDC beim Booten"** ist spezifisch für ESP32-basierte Boards (wie ESP32-S2, S3 oder C3) mit nativer USB-Unterstützung. Sie legt fest, wie Ihr Computer das Board erkennt, sobald es angeschlossen ist.

CDC steht für *Communication Device Class (Kommunikationsgeräteklasse*). Es handelt sich um das Standardprotokoll, das es ermöglicht, ein USB-Gerät als **virtuellen seriellen (COM-)Port zu erkennen** . Dadurch können Sie den `seriellen Monitor` zum Debuggen Ihres Codes verwenden.

Die beiden möglichen Einstellungen:

- **Deaktiviert:** Das Board aktiviert den USB-Seriell-Port beim Start nicht automatisch. Um Meldungen im seriellen Monitor anzuzeigen, benötigen Sie in der Regel eine externe Hardwarekomponente (USB-zu-UART-Konverter) oder müssen USB in Ihrem Code manuell konfigurieren.
- **Aktiviert:** Dies ist die gängigste Einstellung für Anfänger. Sobald die Karte eingeschaltet wird, meldet sie sich beim Computer *als COM-Port . Dadurch können Sie* `Serial.print()` sofort ohne aufwendige Konfiguration verwenden .

Zigbee Mode

Die **Zigbee Mode** -Option wurde mit neueren Chipgenerationen, insbesondere dem **ESP32-H2** und dem **ESP32-C6** , eingeführt. Im Gegensatz zum klassischen ESP32, der nur WLAN und Bluetooth unterstützt, verfügen diese Chips über ein Funkmodul, das mit dem **IEEE 802.15.4-** Standard kompatibel ist .

Mit dieser Option in der Arduino IDE können Sie die Rolle definieren, die Ihr Board innerhalb eines Smart-Home-Netzwerks (Hausautomation) spielen soll.

ZigBee ist die ideale Alternative zu WLAN für vernetzte Objekte (IoT), weil:

- **Extrem niedriger Stromverbrauch:** Ein Sensor kann jahrelang mit einer Knopfzellenbatterie betrieben werden.
- **Mesh-Netzwerk:** Jedes an das Stromnetz angeschlossene Gerät kann das Signal wiederholen, um die Reichweite zu vergrößern.
- **Standard für Hausautomation:** Dies ist die Sprache, die von Philips Hue, IKEA TRÅDFRI oder Xiaomi Sensoren verwendet wird.

Die verschiedenen verfügbaren Modi

Im Menü haben Sie in der Regel die Wahl zwischen drei Gerätetypen:

1. Zigbee-Koordinator

Es ist das „Gehirn" des Netzwerks.

- **Rolle:** Es erstellt das Netzwerk, autorisiert neue Geräte zum Anschluss und verwaltet die Sicherheit.
- **Hinweis:** Pro Zigbee-Netzwerk kann es nur einen Koordinator geben (oft Ihr Smart-Home-Gerät oder ein dedizierter USB-Stick).

2. ZigBee-Router

Es handelt sich um ein Zwischengerät, das permanent mit Strom versorgt wird (z. B. eine Glühbirne oder eine intelligente Steckdose).

- **Rolle:** Es erfüllt seine Aufgaben (Einschalten des Lichts) und leitet gleichzeitig Meldungen von anderen Sensoren an den Koordinator weiter.

3. ZigBee-Endgerät

Dies ist das Gerät, das die meiste Zeit im Schlafmodus ist, um seine Batterie zu schonen (z. B. ein Bewegungs- oder Temperatursensor).

- **Rolle:** Es wacht auf, sendet seine Daten und geht wieder in den Schlafmodus. Es leitet niemals Nachrichten von anderen weiter.

Warum gibt es diese Einstellung in der IDE?

Der ESP32 muss seinen Software-Stack je nach gewähltem Modus unterschiedlich konfigurieren. Wählt man beispielsweise **„Endgerät" ,** enthält die **IDE** Funktionen zur Verwaltung des *Tiefschlafmodus* . Wählt man hingegen **„Router"** , werden Netzwerk-Routing-Funktionen aktiviert.

Diese Chips bilden auch die Grundlage des neuen **Matter- Standards** . Der ZigBee-Modus ist oft der erste Schritt, um eine Verbindung zwischen Ihren alten ZigBee-Geräten und Ihren neuen Matter-Geräten herzustellen.

Warnung: Wenn Sie einen klassischen ESP32 (WROOM-32) besitzen, wird diese Option nicht angezeigt oder hat keine Wirkung, da die Funkhardware nicht kompatibel ist.

ESP32-Platinen

Das ESP32-Ökosystem hat sich enorm diversifiziert. Heute spricht man nicht mehr von einer einzigen „Platine", sondern von einer riesigen Familie von Chips, die an spezifische Bedürfnisse (Leistung, Kosten oder Konnektivität) angepasst sind.

Bevor Sie sich für ein Entwicklungsboard (die Leiterplatte) entscheiden, müssen Sie den darauf befindlichen Chip auswählen.

Serie	Hauptverwendung	Hauptmerkmale
ESP32 (Original)	Vielseitigkeit	Dual-Core-Prozessor, WLAN, Bluetooth Classic + BLE. Dies ist die Basisversion, eine sehr robuste Ausführung.
ESP32-S2	Sicherheit & USB	Einkernprozessor, **nativer USB-Anschluss** , mehr GPIO-Pins, aber **kein Bluetooth** .
ESP32-S3	KI & Video	Dual-Core-Prozessor, nativer USB-Anschluss, KI-Unterstützung, ideal für Sprach- oder Bilderkennung.
ESP32-C3 / C6	Kostengünstig / IoT	Einkernprozessor (RISC-V). Der **C6** unterstützt **Wi-Fi 6** und **Matter** .
ESP32-H2	Mesh-Hausautomation	**Kein WLAN** . Nur Bluetooth und Zigbee/Thread.

Die gängigsten Kartenformate

Sobald der Chip ausgewählt ist, wird er auf einem gebrauchsfertigen Entwicklungsboard montiert. Zu den klassischen (breadboard-kompatiblen) Optionen gehören:

8. **ESP32 DevKit V1 (NodeMCU):** Das beliebteste Modell. Es verfügt über 30 oder 38 Pins, einen Micro-USB-Anschluss und zwei Tasten (Boot und EN). Ideal für Einsteiger.
9. **ESP32-WROOM / WROVER:** Dies sind häufig Modulbezeichnungen. Der *WROVER* verfügt üblicherweise über zusätzlichen Speicher (PSRAM), der für die Bildverarbeitung nützlich ist.

Spezialisierte ESP32-Boards:

- **ESP32-CAM:** Enthält einen kleinen Kamerasensor (OV2640) und einen SD-Kartenleser. Sehr beliebt für Überwachungszwecke.
- **M5Stack:** Platinen in einem kleinen Gehäuse mit Bildschirm, Akku und Grove-Anschlüssen. Es ist das „LEGO" des ESP32.

- **TinyPICO / LOLIN D32:** Miniaturformate für Projekte mit begrenztem Platzangebot.

Moderne ESP32-Boards (USB-C & Kompakt):

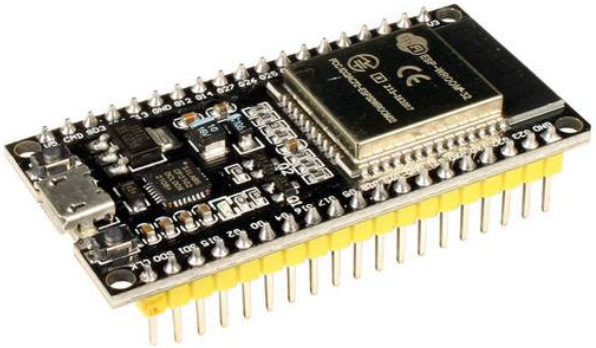
- **XIAO ESP32 (Seeed Studio):** Winzig, etwa so groß wie eine Briefmarke, oft mit USB-C-Anschluss.
- **LilyGO T-Display:** Integriert direkt einen Farb-LCD-Bildschirm, ideal für die Erstellung von Schnittstellen ohne aufwendige Verkabelung.

Welche Option ist die richtige für Ihr Projekt?

- **„Ich bin ein absoluter Anfänger."** Besorg dir ein Standard **-ESP32-Entwicklungskit V1** . Es ist preiswert (ca. 5–8 €) und wird in allen Tutorials im Internet verwendet.
- **„Ich möchte moderne Hausautomation realisieren."** Entscheiden Sie sich für den **ESP32-C6** . Seine Kompatibilität mit dem **Matter- Standard** und Wi-Fi 6 macht ihn zur besten Wahl für die Zukunft.
- **„Ich brauche Gesichts- oder Spracherkennung."** Dann ist der **ESP32-S3 genau das Richtige** . Seine Rechenleistung und seine Vektorbefehle sind dafür wie geschaffen.

ESP32 WROOM

ESP32-WROOM- Board (in seiner Vollversion oft als DevKit V1 bezeichnet) ist der absolute Standard für Entwickler. Es basiert auf dem WROOM-32-Modul, das den originalen Espressif-Chip enthält.



Hier sind die detaillierten technischen Spezifikationen:

Das Herzstück des Systems (Mikrocontroller)

Hier liegt die volle Stärke des Boards. Anders als bei einem klassischen Arduino handelt es sich hier um eine wesentlich robustere Architektur.

4. **Prozessor:** Xtensa® Dual-Core 32-Bit LX6.
5. **Taktfrequenz:** Einstellbar von 80 MHz bis **240 MHz** .
6. **Interner Speicher:** 520 KB SRAM.
7. **Flash-Speicher:** Üblicherweise **4 MB** (zum Speichern Ihres Codes und Ihrer Dateien).

Drahtlose Konnektivität

Das ist die Stärke des ESP32. Er ist von Haus aus "vernetzt".

- **Wi-Fi:** 802.11 b/g/n (bis zu 150 Mbit/s). Kann im Stationsmodus (Verbindung zum Router) oder im Access-Point-Modus (Erstellung eines eigenen Netzwerks) betrieben werden.

- **Bluetooth:** Dual-Modus, unterstützt **Bluetooth Classic** (Audio, Datenübertragung) und **Bluetooth Low Energy (BLE)** (geringer Stromverbrauch, ideal für Sensoren).

Eingänge und Ausgänge (GPIO)

Das DevKit-Board hat typischerweise **30** oder **38 Pins** . Es bietet eine hohe Flexibilität für Sensoren und Aktoren:

4. **ADC (Analog-Digital-Wandler):** Bis zu 18 Kanäle in 12 Bit (sehr präzise).
5. **DAC (Digital-Analog-Wandler):** 2 Kanäle mit 8 Bit zur Erzeugung echter analoger Signale.
6. **Berührungssensoren:** 10 kapazitive Pins (ermöglichen die Herstellung von Berührungstasten ohne externe Komponenten).
7. **Kommunikationsschnittstellen:** 3x UART, 3x SPI, 2x I2C, CAN-Bus 2.0 und I2S (für Audio).
8. **PWM:** 16 unabhängige Kanäle (zur Ansteuerung von LEDs oder Servos).

Energieverbrauch

Der ESP32-WROOM ist dank seiner verschiedenen Standby-Modi für batteriebetriebene Projekte konzipiert:

Mode	Verbrauch (ca.)	Zustand
Aktiv	160 ~ 260 mA	Wi-Fi und CPU laufen.
Modem-Schlafmodus	20 ~ 68 mA	CPU aktiv, WLAN aus.
Leichter Schlaf	0,8 mA	CPU pausiert, schnelles Aufwachen.
Tiefschlaf	**10 µA**	Fast alles ist ausgeschaltet, außer der Echtzeituhr (RTC).

Stromversorgung

4. **Über USB:** 5 V über Micro-USB-Anschluss.
5. **Über den Vin-Pin: Hier** können Sie geregelte 5 V einspeisen.
6. **Logikspannung: Warnung: Der ESP32 arbeitet mit 3,3 V.** Das Anlegen von 5 V an einen Datenpin (GPIO) kann den Chip zerstören.

Hinweis: Das WROOM-Modul verfügt über eine integrierte Leiterplattenantenne (das Zickzackmuster auf der Schaltung), die eine sehr gute Wi-Fi-Reichweite ohne externe Antenne ermöglicht.

Parameter tools für ESP32-WROOM

Hier sind die wichtigsten Parameter für die Konfiguration der *Werkzeugoption* der Arduino IDE vor dem Kompilieren und Hochladen von ESP32forth:

```
Board: ESP32 Dev Module
Port: COMx
```

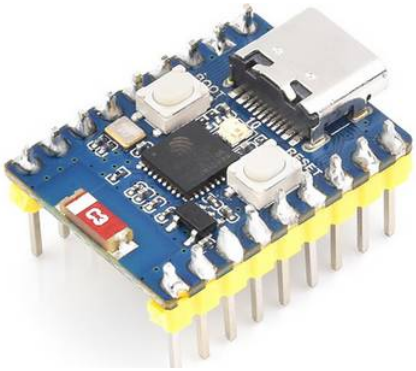
```
---------------------------------
CPU Frequency: 240 Mhz
Core Debug Level: None
Erase All Flash Before Sketch Upload: Disabled
Events Run On: Core 1
Flash Frequency: 80 Mhz
Flash Mode: QIO
Flash Size: 4MB
JTAG Adapter: Disabled
Arduino runs on: Core 1
Partition Scheme: No OTA (2 MB APP / 2 MB SPIFFS)
PSRAM: Disabled
Upload Speed: 921600
ZigBee Mode: Disabled
```

Parameter tools für ESP32-WROVER

Hier sind die wichtigsten Parameter für die Konfiguration der *Werkzeugoption* der Arduino IDE vor dem Kompilieren und Hochladen von ESP32forth:

```
Board: ESP32 Dev Module
Port: COMx
---------------------------------
CPU Frequency: 240 Mhz
Core Debug Level: None
Erase All Flash Before Sketch Upload: Disabled
Events Run On: Core 1
Flash Frequency: 80 Mhz
Flash Mode: QIO
Flash Size: 4MB
JTAG Adapter: Disabled
Arduino Runs On:
Partition Scheme: No OTA (2 MB APP / 2 MB SPIFFS)
PSRAM: Enabled
Upload Speed: 921600
ZigBee Mode: Disabled
```

ESP3 C3 Zero



Der **ESP32-C3 Zero** ist ein ultrakompaktes Entwicklungsboard (oft hergestellt von Waveshare oder ähnlichen Herstellern), das als moderne und wirtschaftliche Alternative zum ESP8266 konzipiert wurde.

Im Gegensatz zum von Ihnen erwähnten ESP32-S3 verwendet der C3 eine Single-Core **-RISC-V-Architektur** . Dadurch eignet er sich ideal für kleine vernetzte Geräte, die keine enorme Rechenleistung benötigen, aber einen geringen Stromverbrauch aufweisen müssen.

Technische Spezifikationen (Hardware)

- **Prozessor:** Einkerniger 32-Bit-RISC-V-Prozessor mit einer Taktfrequenz von bis zu **160 MHz** .
- **Direktzugriffsspeicher (SRAM):** 400 KB intern.

- **Speicher (Flash):** 4 MB integriert.
- **Konnektivität:**
  - Wi-Fi 4 (802.11 b/g/n) mit 2,4 GHz.
  - Bluetooth 5.0 und **BLE (Bluetooth Low Energy)** .
- **Stromverbrauch:** Sehr gering im *Tiefschlafmodus* (ca. **43 µA**).
- **Abmessungen:** Ungefähr 23,5 × 18 mm (Briefmarkengröße).

Schnittstellen und Pins (E/A)

Obwohl die Karte winzig ist, zeigt sie das Wesentliche:

- **15** multifunktionale GPIOs.
- **6 analoge Eingänge (ADC)** auf 12 Bit.
- **Serielle Schnittstellen:** 2x UART, 1x I2C, 3x SPI, 1x I2S.
- **Nativer USB-Anschluss:** USB-C-Anschluss mit **integriertem CDC/JTAG -Controller** (kein Konverterchip wie beim CH340 erforderlich).
- **RGB-LED:** Eine WS2812 (Neopixel) LED ist oft auf der Platine integriert (üblicherweise an **GPIO 8**).

Empfohlene Einstellungen in der Arduino IDE

So passen Sie die Einstellungen für diese spezielle Karte an:

- **Board:** "ESP32C3 Dev Module" (oder "Waveshare ESP32-C3-Zero", falls verfügbar).
- **USB CDC beim Booten: Aktiviert** (unbedingt erforderlich, um den seriellen Port über USB-C zu sehen).
- **Blitzmodus: DIO** (Der QIO-Modus wird von den in diesem kleinen Format integrierten Chips nicht immer unterstützt).
- **Partitionsschema:** „4 MB mit SPIFFS" oder „Minimales SPIFFS", je nach Ihren Bedürfnissen.
- **PSRAM: Deaktiviert** (der ESP32-C3 verfügt nicht über PSRAM und kann dieses nicht ohne Weiteres extern verwalten).

Wichtige Punkte

- **Fehlendes Bluetooth Classic:** Im Gegensatz zu älteren ESP32-Modellen **unterstützt der C3 nur Bluetooth Low Energy (BLE)** . Ohne das BLE-Protokoll kann er nicht als herkömmlicher Bluetooth-Lautsprecher oder HID-Tastatur verwendet werden.
- **Bootloader-Modus:** Falls das Hochladen fehlschlägt, halten Sie die **BOOT- Taste** (oft mit GPIO 9 gekennzeichnet) gedrückt, drücken Sie kurz **RESET** und lassen Sie dann BOOT los. Das Board wird anschließend als serielle Schnittstelle erkannt und kann geflasht werden.

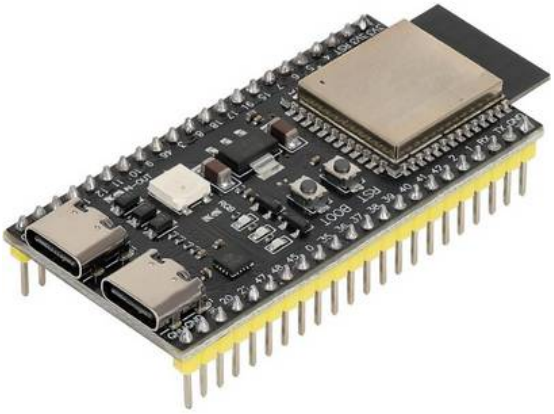
Parametertools für ESP32 C3 Zero

der Werkzeugoptionen der Arduino IDE vor dem Kompilieren und Hochladen von ESP32forth:

```
Board: ESP32C3 Dev Module
Port: COMx
---------------------------------
USB CDC On Boot: Enabled
CPU Frequency: 160 Mhz
Core Debug Level: None or Info
Erase All Flash Before Sketch Upload: Disabled
Flash Frequency: 80 Mhz
Flash Mode: QIO 80 Mhz
Flash Size: 4MB
JTAG Adapter: Disabled
Arduino Runs On:
Partition Scheme: No OTA (2 MB APP / 2 MB SPIFFS)
PSRAM: OPI PSRAM
Upload Speed: 921600
ZigBee Mode: Disabled
```

ESP32 S3 WROOM / Entwicklermodul

Der **ESP32-S3-WROOM** ist eines der leistungsstärksten Module der Espressif-Familie. Er ist der leistungsstärkere Nachfolger des klassischen ESP32-WROOM und wurde speziell für **Künstliche Intelligenz (AIoT)** und Signalverarbeitung optimiert.



Hier sind die wichtigsten Informationen, die Sie wissen müssen:

Rechenleistung und KI

Im Gegensatz zum klassischen ESP32 (LX6-Kern) verwendet der S3 die **Xtensa® LX7- Architektur**
.

3. **Prozessor:** Dual-Core 32-Bit, bis zu **240 MHz** .
4. **KI-Beschleunigung:** Es verfügt über spezifische Anweisungen zur Beschleunigung von Berechnungen neuronaler Netze (Spracherkennung, Gesichtserkennung).
5. **Speicher:** Es verfügt über **512 KB internen SRAM** , wird aber oft von **PSRAM** (externer RAM bis zu 16 MB) begleitet, um rechenintensive Projekte (hochauflösende Bildschirme, Audio usw.) zu bewältigen.

Konnektivität

- **Wi-Fi:** 802.11 b/g/n (2,4 GHz).
- **Bluetooth:** Version **5.0 LE (Low Energy)** . *Hinweis: Im Gegensatz zum ursprünglichen ESP32 unterstützt er nicht mehr Bluetooth „Classic", sondern nur noch LE.*

- **Nativer USB-Anschluss:** Dies ist eine seiner größten Stärken. Es verfügt über einen **USB-OTG- Controller** , was bedeutet, dass es von einem PC ohne zusätzlichen Konverterchip nativ als Tastatur, Maus oder USB-Stick erkannt werden kann.

Die verschiedenen Versionen

Der vollständige Name sieht oft so aus: *ESP32-S3-WROOM-1-N16R8* . So entschlüsseln Sie ihn:

- **N4, N8, N16:** Flash-Speichergröße (4, 8 oder 16 MB).
- **R2, R8:** Größe des PSRAM (2 oder 8 MB). Fehlt das „R", ist kein zusätzlicher RAM vorhanden.
- **WROOM-1:** Integrierte Leiterplattenantenne.
- **WROOM-1U:** Anschluss für externe Antenne (U.FL).

Vergleich:

Merkmal	ESP32 (Original)	ESP32-S3
Herzen	LX6	**LX7 (neuer)**
KI-Anweisungen	NEIN	**Ja**
Bluetooth	4.2 + Klassisch	**5.0 LE + Mesh**
Native USB	NEIN	**Ja (OTG)**
GPIO	~34	**~36 (flexibler)**

Wenn Sie Spracherkennung in Betracht ziehen, einen komplexen Bildschirm ansteuern möchten oder für ein Projekt einen nativen USB-Anschluss benötigen, ist dies die ideale Karte.

Parametertools für ESP32-S3-WROOM

Hier sind die wichtigsten Parameter für die Konfiguration der *Werkzeugoption* der Arduino IDE vor dem Kompilieren und Hochladen von ESP32forth:

```
Board: ESP32S3 Dev Module
Port: COMx
---------------------------------
USB CDC On Boot: Enabled
CPU Frequency: 240 Mhz
Core Debug Level: None
USB DFU On Boot: Disabled
Erase All Flash Before Sketch Upload: Disabled
Events Run On: Core 1
Flash Mode: QIO 80 Mhz
Flash Size: 4MB
JTAG Adapter: Disabled
Arduino Runs On: Core 1
Partition Scheme: No OTA (2 MB APP / 2 MB SPIFFS)
PSRAM: OPI PSRAM QPSI PSRAM
Upload Mode: UART0 / Hardware CDC
Upload Speed: 921600
```

```
USB Mode: USB-OTG (TinyUSB)
ZigBee Mode: Disabled
```

Parametertools für ESP32-S3-WROOM-N16R8

Diese ESP32-S3-Version verfügt über mehr Speicher:

- 16 MB Flash-Speicher
- 8 MB PSRAM-Speicher

Hier sind die wichtigsten Parameter für die Konfiguration der *Werkzeugoption* der Arduino IDE vor dem Kompilieren und Hochladen von ESP32forth für dieses spezielle Board:

```
Board: ESP32S3 Dev Module
Port: COMx
---------------------------------
USB CDC On Boot: Disabled
CPU Frequency: 240 Mhz
Core Debug Level: None
USB DFU On Boot: Disabled
Erase All Flash Before Sketch Upload: Disabled
Events Run On: Core 1
Flash Mode: QIO 80 Mhz
Flash Size: 16MB
JTAG Adapter: Disabled
Arduino Runs On: Core 1
USB Firmware MSC On Boot: Disabled
Partition Scheme: 16M Flash (3MB No OTA / 1MB SPIFFS)
PSRAM: OPI PSRAM QPSI PSRAM
Upload Mode: UART0 / Hardware CDC
Upload Speed: 921600
USB Mode: Hardware CDC and JTAG
ZigBee Mode: Disabled
```

Ressourcen

- Offizielle ARDUINO IDE-Website
  https://www.arduino.cc/en/software/

Forth

- Webseite von Brad Nelson, dem Entwickler von ESP32forth, auf Englisch
  https://esp32forth.appspot.com/ESP32forth.html
- Marc PETREMANNs Website, auf Französisch und Englisch
  https://esp32.arduino-forth.com/
- Forth Standard-Website, auf Englisch
  https://forth-standard.org/